

Partie 1 : Jeu de Pacman simplifié

1.1 Pour commencer

Conformément à l'énoncé, l'état du système est défini à partir de la position du Pacman et de l'état des nourritures présentes dans l'environnement. On note F_i la variable binaire associée à la nourriture numéro i , avec $i \in [0, n - 1]$, telle que $F_i = 0$ lorsque la nourriture est intacte et $F_i = 1$ lorsqu'elle a été consommée. La position du Pacman est représentée par la variable Pos , correspondant à l'indice de la case occupée sur le damier, avec la convention $Pos = 0$ pour la case située en bas à gauche.

Le calcul de l'état global est obtenu à l'aide de la formule suivante :

$$State = Pos + N_{Pos} \times \sum_{i=0}^{n-1} F_i \times 2^i$$

où N_{Pos} désigne le nombre total de positions possibles sur le damier et n le nombre de nourritures.

Ce codage permet de représenter l'état complet du jeu, car il distingue à la fois toutes les positions possibles du Pacman et toutes les combinaisons possibles des états des nourritures, mangées ou non. Chaque configuration physique du jeu est ainsi associée à un état unique, ce qui garantit une représentation exhaustive de l'espace d'état et satisfait la propriété de Markov requise pour l'apprentissage par renforcement.

En utilisant cette formule et en se basant sur l'environnement décrit par le fichier *board0.txt*, qui correspond à un damier minimaliste de taille 2×2 comportant une unique nourriture, on obtient :

$$State = Pos + (2 \times 2) \times F_0 \times 2^0 = Pos + 4 \times F_0$$

Le nombre théorique total d'états est alors donné par le produit du nombre de positions possibles et du nombre de configurations des nourritures, soit :

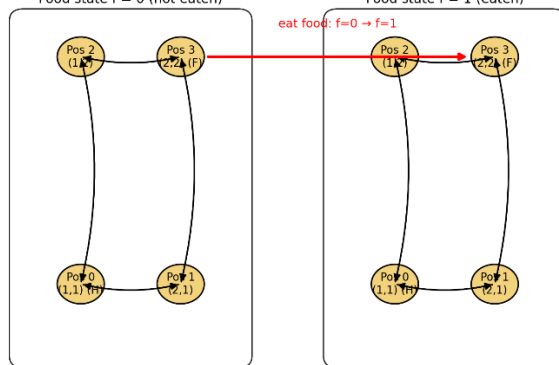
$$N_{State} = N_{Pos} \times 2^n = 4 \times 2 = 8$$

Cependant, l'exploration de l'environnement en pratique montre que tous ces états théoriques ne sont pas nécessairement atteignables. En effet, lorsque le Pacman se déplace sur la case contenant la nourriture, celle-ci est automatiquement consommée, ce qui empêche certaines configurations de persister dans le temps. De plus, le jeu se termine dès que la nourriture a été consommée et que le Pacman rejoint sa maison. Ces configurations correspondent à des états terminaux, également

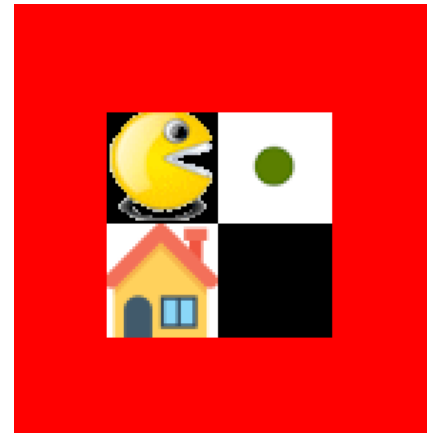
appelés états absorbants, qui interrompent immédiatement l'épisode et limitent l'exploration de l'espace d'état.

Bien que l'espace d'état théorique comporte 8 états, la dynamique du jeu induit des transitions irréversibles, notamment lors de la consommation de la nourriture. Une fois la nourriture consommée, il est impossible de revenir à un état où celle-ci est intacte. De plus, l'état correspondant au Pacman à sa maison avec toute la nourriture consommée est un état absorbant qui termine immédiatement l'épisode. Ainsi, si tous les états sont théoriquement définis, la chaîne de Markov associée n'est pas ergodique et certains états ne peuvent être visités qu'un nombre limité de fois au cours d'un épisode.

Figure — Markov chain representation for board0.txt (2x2, H at (1,1), F at (2,2))



State space split by food status. Within-layer arrows represent valid moves; red arrow is the irreversible food consumption transition.



1.2 Apprentissage de la table action-value function (Q)

La fonction *Qlearning* implémente l'algorithme de Q-learning par différence temporelle (Temporal Difference), permettant d'estimer la fonction action-valeur $Q(s, a)$ sans connaissance explicite de la dynamique de transition de l'environnement. À chaque interaction, l'estimation de $Q(s, a)$ est mise à jour à partir de la récompense immédiate observée et de la meilleure valeur Q associée à l'état suivant, ce qui permet une propagation progressive de l'information sur les récompenses futures.

La politique ϵ -greedy assure un compromis entre exploration et exploitation : avec une probabilité ϵ , une action aléatoire est sélectionnée afin d'explorer l'environnement, tandis qu'avec une probabilité $1 - \epsilon$, l'action maximisant la valeur de Q est choisie.

B-2 Apprentissage sur le fichier *board1*

Le fichier *board1.txt* décrit un environnement de taille 3×3 comportant une nourriture et un poison. Le nombre de positions possibles pour le Pacman est donc :

$$N_{pos} = 3 \times 3 = 9$$

La nourriture étant décrite par une variable binaire (consommée ou non), le nombre total d'états est :

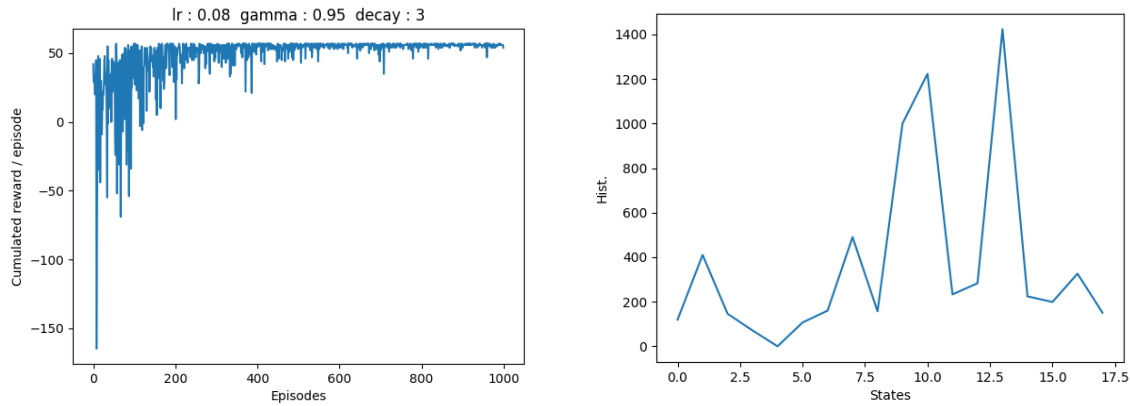
$$N_{state} = 9 \times 2 = 18$$

La table Q estimée est ainsi de dimension 18×4 , correspondant à l'ensemble des états possibles et aux quatre actions disponibles (bas, gauche, haut, droite).

Après apprentissage, les statistiques calculées sur 100 parties montrent une récompense cumulée moyenne de 56.46, un taux de victoire de 100 %, un nombre moyen de déplacements de 4.54 et aucun passage sur une case poison. Ces résultats indiquent que le Pacman a appris une stratégie efficace, lui permettant de consommer la nourriture puis de rejoindre sa maison en un nombre réduit de déplacements, tout en évitant systématiquement la case poison.

L'évolution de la récompense cumulée au fil des épisodes présente une forte variabilité au début de l'apprentissage, ce qui s'explique par la phase d'exploration durant laquelle les actions sont majoritairement aléatoires. À mesure que le nombre d'épisodes augmente, la récompense cumulée se stabilise autour d'une valeur élevée, traduisant la convergence progressive de la politique vers une stratégie quasi optimale. Quelques baisses ponctuelles subsistent, dues à l'exploration résiduelle imposée par la politique ϵ -greedy.

L'histogramme des états visités met en évidence une fréquentation non uniforme de l'espace d'état. Cette observation s'explique par le fait que la politique apprise privilégie des trajectoires courtes reliant les positions initiales à la nourriture puis à la maison, tout en évitant la case poison. En phase d'exploitation, seuls les états appartenant à ces trajectoires optimales sont majoritairement visités, tandis que les autres deviennent rares voire quasiment inaccessibles.



B-3 Influence du facteur d'actualisation γ

Lorsque l'on prend $\gamma = 1$, les récompenses futures ne sont pas actualisées. L'agent cherche donc à maximiser la somme totale des récompenses jusqu'à la fin de l'épisode. La courbe d'apprentissage montre une forte variabilité initiale liée à l'exploration, puis une stabilisation autour d'une récompense cumulée élevée. Les statistiques sur 100 parties indiquent une récompense moyenne d'environ 56.4, un taux de victoire de 100 %, un nombre moyen de déplacements proche de 4.6 et l'absence totale de passage sur la case poison.

La matrice Q obtenue contient des valeurs élevées pour les actions appartenant aux trajectoires optimales. Ces valeurs ne représentent pas des récompenses immédiates, mais des estimations du retour cumulatif attendu à partir d'un état-action donné. Elles ne correspondent pas à des valeurs théoriques uniques, car le retour cumulé dépend du nombre effectif de déplacements avant la fin de l'épisode, lequel peut varier selon la position initiale et l'exploration résiduelle. De plus, certains états étant peu visités, leurs valeurs de Q restent partiellement estimées.

Lorsque l'on prend $\gamma = 0.1$, les récompenses futures sont fortement décotées et l'agent privilégie davantage le gain immédiat. La courbe d'apprentissage et les statistiques finales restent globalement similaires, avec un taux de victoire de 100 % et une absence totale de poison. En revanche, la matrice Q est sensiblement différente : la majorité des valeurs sont proches des récompenses immédiates, souvent négatives en raison du coût de déplacement, et seules les actions menant directement à la nourriture ou à la victoire conservent des valeurs élevées.

Cette différence entre les matrices Q est une conséquence directe du rôle du facteur d'actualisation γ . Dans ce problème simple, les épisodes sont courts et la stratégie optimale est peu ambiguë, ce qui explique que la politique finale soit similaire pour $\gamma = 1$ et $\gamma = 0.1$. En revanche, dans des environnements plus complexes nécessitant des sacrifices à court terme pour un gain futur, le choix de γ aurait une influence beaucoup plus marquée sur la politique apprise.

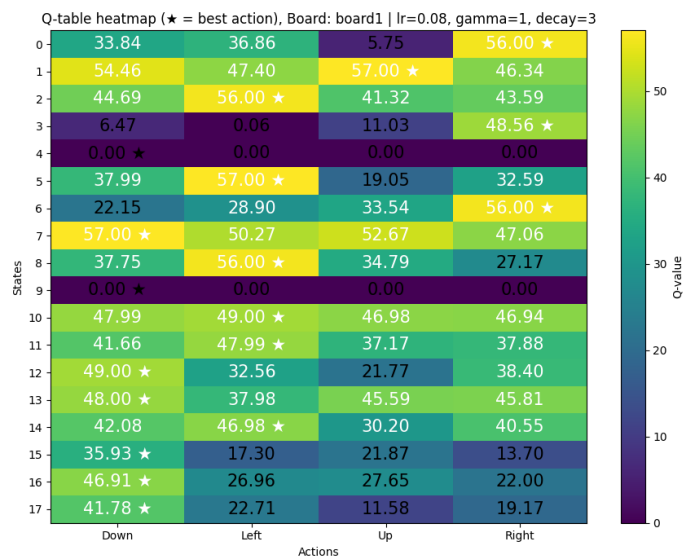


Figure 1. Q-Table board 1, gamma = 1

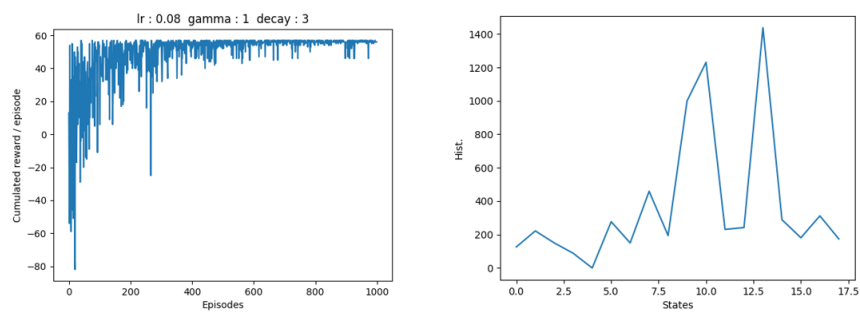


Figure 2. Courbe apprentissage et fréquence de visiste des états, gamma = 1

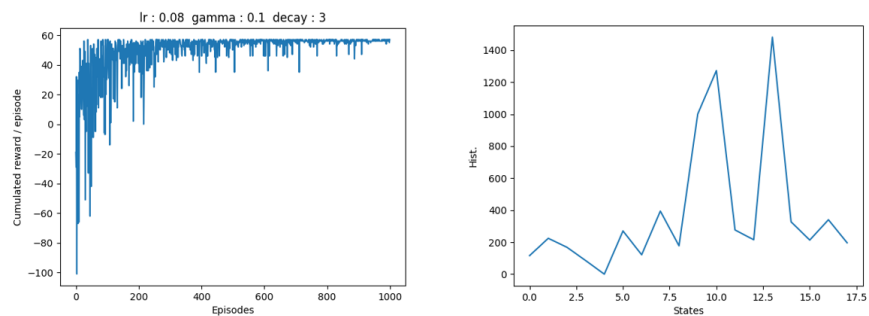


Figure 3. . Courbe apprentissage et fréquence de visiste des états, gamma = 0.1

B-4 Influence d'un facteur d'actualisation nul ($\gamma = 0$)

Lorsque l'on prend $\gamma = 0$, les récompenses futures ne sont plus prises en compte dans la mise à jour de la fonction action-valeur. L'agent optimise alors uniquement la récompense immédiate associée à chaque action, sans chercher à anticiper les conséquences à long terme de ses décisions. Le problème d'apprentissage se réduit ainsi à une forme de décision myope, dans laquelle chaque action est évaluée indépendamment de la trajectoire globale.

La courbe d'apprentissage obtenue reflète clairement ce comportement. La récompense cumulée présente une forte variabilité sur l'ensemble des épisodes et ne converge pas vers un plateau élevé, contrairement aux cas $\gamma = 1$ et $\gamma = 0.1$. Cette instabilité s'explique par le fait que l'agent n'est pas en mesure de valoriser les actions qui permettent de se rapprocher progressivement de la nourriture ou de la maison, lorsque ces objectifs ne procurent pas de récompense immédiate.

Les statistiques calculées sur 100 parties confirment cette analyse : la récompense cumulée moyenne chute à 24.88, le taux de victoire tombe à 70 %, et le nombre moyen de déplacements augmente fortement pour atteindre environ 18.1. L'agent adopte ainsi des trajectoires longues et inefficaces, errant fréquemment dans l'environnement sans construire une stratégie cohérente menant systématiquement à la victoire.

Dans ce contexte, l'exploration ne permet pas une amélioration progressive de la politique, car l'information sur les récompenses futures n'est jamais propagée aux états précédents. Même lorsque l'agent découvre ponctuellement une trajectoire gagnante, celle-ci n'est pas valorisée dans les états intermédiaires, ce qui empêche toute généralisation.

Ces résultats illustrent de manière claire le rôle fondamental du facteur d'actualisation γ dans l'apprentissage par renforcement. Lorsque $\gamma = 0$, l'agent perd toute capacité d'anticipation, ce qui conduit à une politique sous-optimale, instable et peu performante, y compris dans un environnement simple. Le facteur d'actualisation apparaît ainsi comme un élément indispensable à l'apprentissage de stratégies efficaces sur des épisodes multi-étapes.

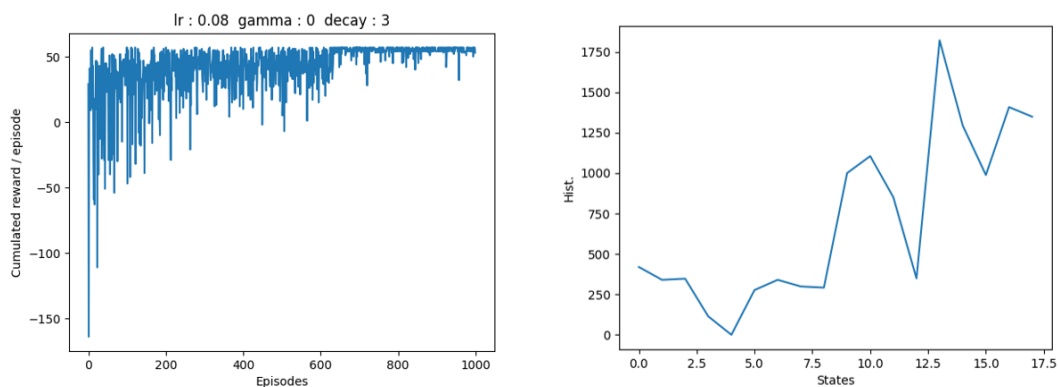


Figure 4. . Courbe apprentissage et fréquence de visite des états, $\gamma = 0$

B-5 Influence du choix des rewards

Afin d'étudier l'influence des récompenses sur la politique apprise, on fixe les hyperparamètres d'apprentissage, en particulier le facteur d'actualisation $\gamma = 0.95$, et l'on ne fait varier que la pénalité associée au déplacement (*STEP_REWARD*) dans l'environnement *board1*. Ce paramètre joue un rôle central, car il est appliqué à chaque transition et influence directement la longueur des trajectoires privilégiées par l'agent.

Avec *STEP_REWARD* = -0.1 , on obtient une récompense cumulée moyenne de 59.64, un taux de victoire de 100 %, un nombre moyen de déplacements de 4.56 et aucun passage sur la case poison. Le coût associé aux déplacements étant faible, la politique optimale reste inchangée : l'agent apprend à rejoindre la nourriture puis à rentrer à la maison en empruntant un chemin court. La récompense finale est toutefois plus élevée, car chaque déplacement pénalise peu le score total.

Lorsque l'on prend *STEP_REWARD* = -10 , le taux de victoire reste de 100 % et le nombre moyen de déplacements demeure proche de 4.6, indiquant que la stratégie apprise est toujours correcte et minimise la longueur du trajet. En revanche, la récompense cumulée moyenne chute à 24.0, car chaque déplacement est fortement pénalisé. L'agent suit donc une trajectoire quasi identique, mais le coût élevé des actions intermédiaires dégrade fortement le score global.

À l'inverse, lorsque *STEP_REWARD* = $+10$, on observe un comportement pathologique. La récompense cumulée moyenne atteint 500.0, le taux de victoire tombe à 0 %, le nombre moyen de déplacements augmente fortement pour atteindre 50.0, et aucun passage sur la case poison n'est observé. Dans ce cas, l'agent est incité à maximiser le nombre de déplacements plutôt qu'à terminer l'épisode, car chaque action rapporte une récompense positive supérieure à celle associée à la victoire. La politique apprise est alors optimale au sens de la fonction de récompense, mais totalement contraire à l'objectif réel du jeu.

Ces expériences illustrent clairement l'importance du choix des rewards en apprentissage par renforcement. Une fonction de récompense mal définie peut conduire l'agent à adopter des comportements non souhaités, voire absurdes du point de vue de la tâche à accomplir. Le reward ne se contente pas de mesurer la performance : il définit implicitement l'objectif que l'agent cherche à optimiser.

En complément, il aurait été possible d'étudier l'influence des autres composantes de la fonction de récompense, telles que *FOOD_REWARD*, *POISON_REWARD* ou *WIN_REWARD*, afin d'analyser plus finement leur impact sur la politique apprise et sur la dynamique d'apprentissage. Cette étude n'a toutefois pas été menée dans le cadre de ce TP, principalement en raison du temps d'exécution nécessaire pour réaliser des expérimentations systématiques et comparables. Une telle analyse constituerait néanmoins une extension pertinente de ce travail, permettant d'approfondir la compréhension du rôle du reward shaping dans des environnements d'apprentissage par renforcement.

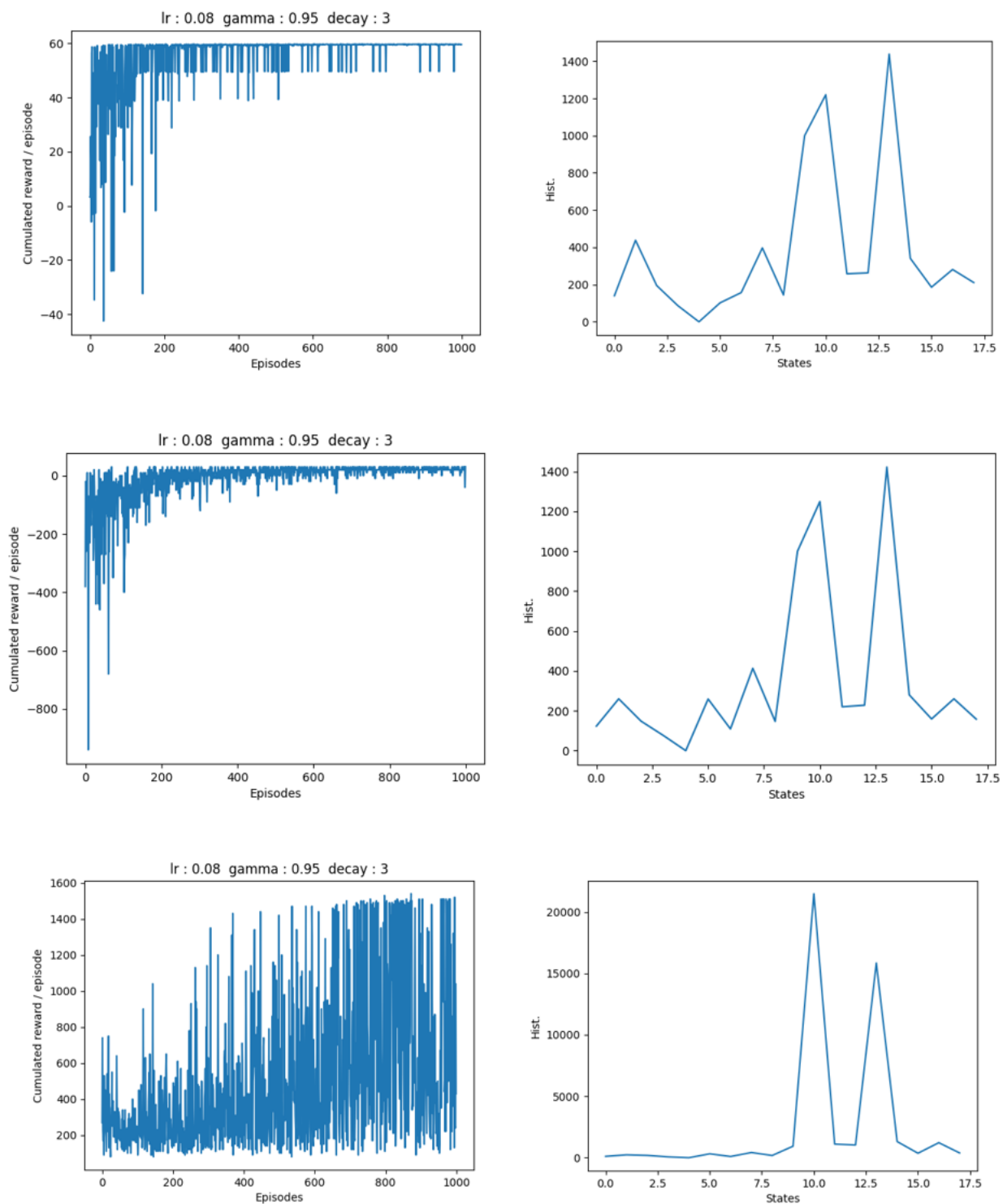


Figure 5. Courbes d'apprentissage et fréquences de visite des états, en fonction du coût de déplacement $(-0.1, -10, +10)$

B-6 Nombre d'états pour la board2 (version allégée recommandée)

La configuration *board2* correspond à un damier de taille 5×5 , soit 25 positions possibles pour le Pacman. On y observe deux cases contenant de la nourriture, chacune décrite par une variable binaire indiquant si elle a été consommée ou non. Conformément au codage de l'état introduit précédemment, et comme les cases poison ne font pas partie de la description de l'état, le nombre total d'états est donné par le produit du nombre de positions et du nombre de configurations possibles des nourritures.

On obtient ainsi un total de 100 états théoriques pour *board2*. Ce résultat met en évidence l'augmentation rapide de la taille de l'espace d'état lorsque la tâche est complexifiée, en particulier par l'ajout de nouvelles nourritures, dont l'effet est exponentiel. Comme dans les configurations précédentes, tous ces états ne sont pas nécessairement atteignables en pratique en raison des transitions irréversibles et de la présence d'états terminaux.

B-7 Rôle du paramètre *exploration_decreasing_decay*

Le paramètre *exploration_decreasing_decay* contrôle la vitesse à laquelle le taux d'exploration décroît au cours de l'apprentissage, et donc la transition progressive entre une phase d'exploration majoritairement aléatoire et une phase d'exploitation de la politique apprise. Une valeur faible de ce paramètre implique une exploration prolongée, tandis qu'une valeur élevée conduit à une exploitation rapide des premières estimations de la fonction Q .

Pour une valeur faible du paramètre (par exemple $decay = 1$ ou 0.1), l'agent conserve une forte probabilité d'exploration sur un grand nombre d'épisodes. La courbe d'apprentissage est alors très bruitée et la convergence visuelle est lente, car l'agent continue à tester des actions sous-optimales même lorsque de bonnes trajectoires ont déjà été identifiées. Néanmoins, les statistiques finales montrent un taux de victoire de 100 %, un nombre moyen de déplacements compris entre 14 et 15, et un nombre moyen de passages sur des cases poison faible, indiquant que la politique finale reste correcte malgré l'instabilité apparente de la courbe.

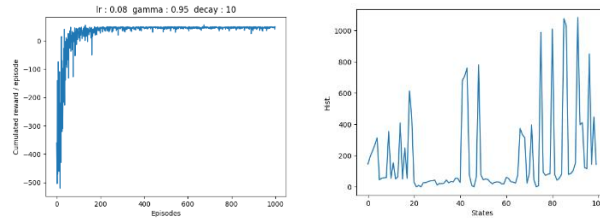
À l'inverse, pour des valeurs plus élevées ($decay = 10$ ou 100), l'exploration décroît rapidement et l'agent exploite plus tôt sa politique. La récompense cumulée se stabilise alors plus rapidement, avec une variance plus faible. Les performances finales restent toutefois très proches de celles obtenues avec une décroissance plus lente, tant en termes de taux de victoire que de longueur moyenne des trajectoires.

Le fait que la table Q soit correctement apprise pour une large gamme de valeurs de *exploration_decreasing_decay* s'explique par plusieurs caractéristiques du problème. D'une part, l'environnement est relativement simple et le nombre de trajectoires réellement pertinentes est limité. D'autre part, le signal de récompense est fortement informatif : les pénalités liées aux déplacements et aux poisons, ainsi que les récompenses associées à la nourriture et à la victoire, permettent d'identifier rapidement les actions favorables. Une fois une trajectoire efficace découverte, la mise à jour par différence temporelle propage cette information aux états précédents, ce qui stabilise progressivement la politique même en présence d'exploration résiduelle.

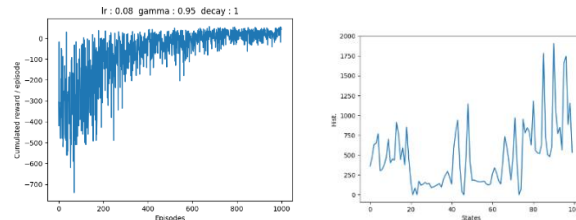
Ainsi, tant que l'exploration n'est pas supprimée de manière excessive dès les premiers épisodes, la convergence vers une politique satisfaisante est robuste au choix du paramètre

exploration_decreasing_decay dans cet environnement. Dans des problèmes plus complexes ou présentant plusieurs stratégies concurrentes, ce paramètre jouerait en revanche un rôle beaucoup plus critique.

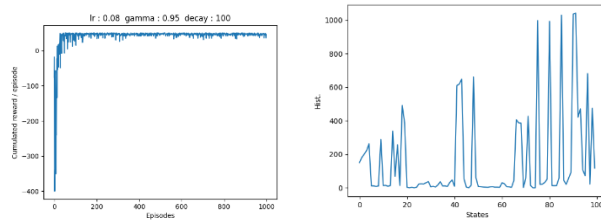
Calcul des stat sur 100 parties
Récompense cumulée moyenne : 46.79 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 14.21
Nombre moyen de cases poison par partie 1.0



Calcul des stat sur 100 parties
Récompense cumulée moyenne : 49.58 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 14.32
Nombre moyen de cases poison par partie 0.71



Calcul des stat sur 100 parties
Récompense cumulée moyenne : 47.22 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 13.78
Nombre moyen de cases poison par partie 1.0



Calcul des stat sur 100 parties
Récompense cumulée moyenne : 48.59 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 15.01
Nombre moyen de cases poison par partie 0.74

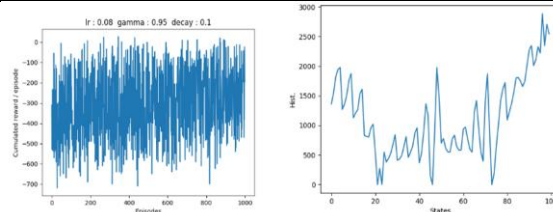


Figure 6. Courbes d'apprentissage et fréquences de visite des états, en fonction du coût de l'*exploration_decreasing_decay*

B-8 Influence des rewards et prévention de l’empoisonnement

Afin d’éviter que le Pacman ne passe par des cases poison, il est nécessaire que la pénalité associée au poison (*POISON_REWARD*) soit suffisamment forte par rapport aux autres composantes de la fonction de récompense, en particulier le coût de déplacement (*STEP_REWARD*) et les récompenses liées à la nourriture ou à la victoire. En apprentissage par renforcement, l’agent ne cherche pas à éviter le poison en tant que tel, mais à maximiser la récompense cumulée attendue.

Si la valeur absolue du *POISON_REWARD* est trop faible, l’agent peut considérer qu’un passage ponctuel par une case poison est acceptable s’il permet de réduire significativement le nombre de déplacements, et donc de limiter l’accumulation de pénalités liées aux pas intermédiaires. Dans ce cas, l’empoisonnement peut devenir une option optimale du point de vue de la fonction de récompense, même s’il est contraire à l’objectif du jeu.

À l’inverse, lorsque la pénalité associée au poison est suffisamment élevée, son impact négatif domine toute stratégie exploitant un raccourci via une case poison. L’agent apprend alors à éviter systématiquement ces cases, quitte à emprunter des trajectoires plus longues. De manière générale, il est nécessaire que le coût du poison soit supérieur au gain maximal pouvant être obtenu en raccourcissant le chemin, c’est-à-dire qu’il dépasse la somme actualisée des pénalités de déplacement évitées et des récompenses intermédiaires potentielles.

Ce réglage illustre l’importance du *reward shaping* : pour obtenir un comportement conforme à l’objectif du jeu, la fonction de récompense doit être conçue de sorte que les comportements indésirables, tels que l’empoisonnement, soient strictement dominés en termes de récompense cumulée, indépendamment de la politique suivie.

B-9 Apprentissage sur la board3 avec les paramètres précédents

En réutilisant les paramètres standards du TP, l’apprentissage sur *board3* permet d’obtenir une politique globalement efficace, bien que moins performante que sur les environnements plus simples. Les statistiques calculées sur 100 parties indiquent une récompense cumulée moyenne de 51.48, un taux de victoire de 99 %, aucune traversée de case poison en moyenne, et un nombre de déplacements relativement élevé (38.62).

La courbe d’apprentissage montre une convergence plus lente et plus bruitée, traduisant une phase d’exploration prolongée et une stabilisation progressive de la politique. Cette dynamique est cohérente avec l’augmentation de la complexité de l’environnement, qui induit un espace d’état plus large et un nombre accru de trajectoires possibles. L’agent apprend ainsi à atteindre l’objectif dans la quasi-totalité des cas, mais la politique apprise reste moins efficace en termes de longueur des trajectoires, ce qui limite la récompense cumulée moyenne.

Ces résultats indiquent que, avec ce paramétrage, le jeu est appris de manière fiable mais pas entièrement optimisée, et qu’un ajustement des hyperparamètres ou un nombre d’épisodes plus important serait nécessaire pour améliorer l’efficacité des trajectoires.

Calcul des stat sur 100 parties
Récompense cumulée moyenne : 51.48 Pourcentage victoire : 99.0 %
Nombre de déplacements moyens 38.62
Nombre moyen de cases poison par partie 0.0

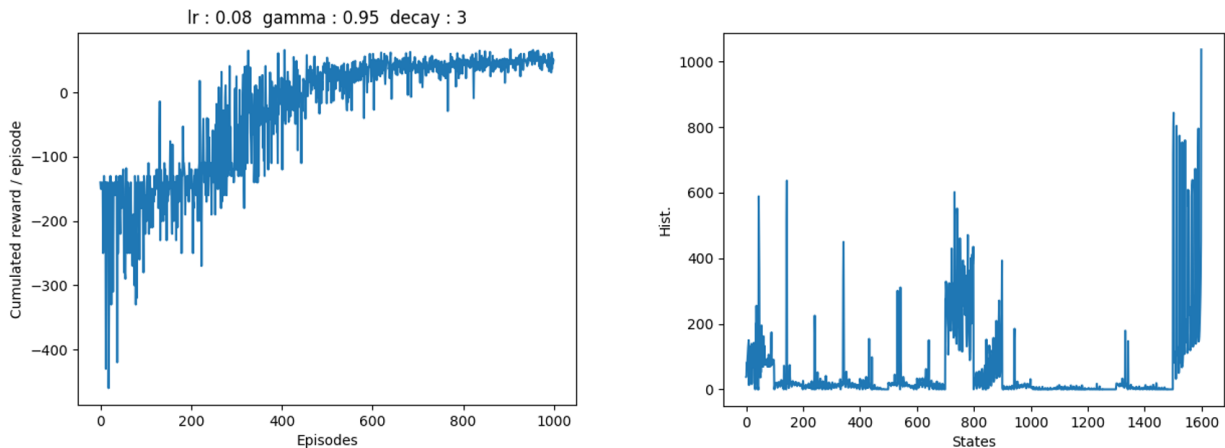


Figure 7. Courbes d'apprentissage et fréquences de visite des états pour la board 3 (paramètres standards)

B-10 Changement de l'emplacement de la maison

Lorsque l'on modifie l'emplacement de la maison, il est en général nécessaire de refaire un apprentissage. En effet, la position de la maison intervient directement dans la dynamique de l'environnement : elle définit la condition de fin de l'épisode ainsi que la récompense associée à la victoire. Changer son emplacement modifie donc la fonction de transition et la fonction de récompense du problème.

La table Q apprise encode la valeur des actions en fonction des états rencontrés, en tenant compte des récompenses futures attendues. Or, ces valeurs dépendent fortement de la distance à la maison et des trajectoires optimales permettant de l'atteindre après avoir consommé toutes les nourritures. Un déplacement de la maison modifie ces trajectoires optimales et rend la table Q apprise précédemment incohérente avec le nouvel objectif.

Dans certains cas simples, la politique issue de l'ancien apprentissage peut rester partiellement exploitable, par exemple si la maison est déplacée vers une position proche de l'ancienne. Toutefois, rien ne garantit que cette politique soit optimale, ni même satisfaisante. Pour garantir un comportement correct et cohérent avec le nouvel environnement, il est donc nécessaire de relancer un apprentissage afin que les valeurs de Q s'adaptent à la nouvelle configuration.

Partie 2 : Deep Q Networks (DQN)

A/ Analyse du fichier *DQNPYtorch.py*

Le fichier *DQNPYtorch.py* met en œuvre un algorithme de Deep Q-Learning dans lequel la fonction action-valeur $Q(s, a)$ est approximée par un réseau de neurones paramétré Q_θ . À partir d'un vecteur d'état $s \in \mathbb{R}^{2+n_{food}}$, le réseau produit en une passe avant un vecteur $Q_\theta(s, \cdot)$ contenant une estimation de la valeur pour chacune des actions disponibles. L'agent sélectionne ensuite une action via une politique ϵ -greedy : avec une probabilité ϵ l'action est tirée aléatoirement (exploration), sinon l'agent choisit l'action maximisant $Q_\theta(s, a)$ (exploitation). Le taux d'exploration décroît au cours de l'apprentissage jusqu'à un minimum fixé.

Chaque interaction avec l'environnement génère une transition (s, a, r, s', d) , où d indique si l'épisode est terminé. Ces transitions sont stockées dans une mémoire de type Replay Buffer, puis échantillonnées aléatoirement sous forme de mini-batches pour l'apprentissage. Ce mécanisme réduit les corrélations temporelles entre observations successives et permet de se rapprocher de l'hypothèse i.i.d. requise par l'optimisation stochastique.

L'apprentissage repose sur une mise à jour par différence temporelle : pour chaque transition échantillonnée, une cible est construite à partir de la récompense immédiate et de l'estimation de la valeur du prochain état, typiquement sous la forme :

$$y = r + \gamma(1 - d) \max_{a'} Q_{\theta^-}(s', a')$$

où Q_{θ^-} désigne le réseau cible (*target network*). Le réseau principal Q_θ est ensuite entraîné en minimisant une perte de régression (par exemple MSE/Huber) entre $Q_\theta(s, a)$ et y , via descente de gradient (par exemple Adam). Le réseau cible est mis à jour périodiquement à partir du réseau principal, ce qui stabilise l'apprentissage en évitant que la cible varie trop rapidement au cours de l'optimisation.

L'entraînement est structuré par épisodes, chacun étant borné par un nombre maximal de pas. Le suivi du reward cumulé par épisode permet d'évaluer la progression de l'apprentissage et la qualité de la politique apprise.

B/ Replay Buffer, réseau target et comparaison DQN / Q-table

Le **Replay Buffer** sert à stocker les transitions (s, a, r, s', d) générées au fil des épisodes et à entraîner le réseau de neurones sur des mini-batches tirés aléatoirement dans cette mémoire, plutôt que sur les transitions les plus récentes uniquement. Dans l'implémentation fournie, chaque interaction ajoute une transition via `memory.add(...)`, puis la méthode `learn()` échantillonne un mini-batch à l'aide de `memory.sample()`. Ce rééchantillonnage aléatoire casse les corrélations fortes entre états successifs et permet de réutiliser des expériences plus anciennes, ce qui rend l'apprentissage plus stable que dans un schéma strictement en ligne.

La terminalité est intégrée sous la forme d'un masque binaire, avec $\text{done} = 1 - d$, ce qui annule le terme de bootstrap dans la cible temporelle lorsque l'épisode est terminé. Ainsi, aucune valeur future n'est propagée à partir d'un état terminal.

Le **réseau target** joue un rôle central dans la stabilisation de l'apprentissage. Les valeurs $Q(s, a)$ à ajuster sont produites par le réseau principal, tandis que la composante correspondant à la valeur future est calculée à l'aide d'un réseau cible distinct. Ce réseau cible est initialisé comme une copie du réseau principal, puis mis à jour périodiquement (toutes les `TARGET_UPDATE_FREQ` itérations) par copie des poids. Cette séparation évite que la cible utilisée pour l'apprentissage ne dépende instantanément des paramètres en cours d'optimisation, ce qui limite les oscillations et les instabilités de convergence.

Pour *board1*, l'état est représenté par un vecteur de dimension $2 + n_{\text{food}}$. Comme il n'y a qu'une seule nourriture, l'entrée du réseau est de dimension 3, et la sortie est de dimension 4 (quatre actions). Avec une architecture MLP composée de deux couches cachées fully-connected de 16 neurones chacune, le nombre total de paramètres (poids et biais) est :

$$(3 \times 16 + 16) + (16 \times 16 + 16) + (16 \times 4 + 4) = 404$$

À titre de comparaison, dans la partie 1 (codage scalaire), le nombre d'états pour un damier 3×3 avec une nourriture est $9 \times 2 = 18$, ce qui conduit à une Q-table de taille $18 \times 4 = 72$ valeurs. Cette comparaison met en évidence la différence entre les deux approches : la méthode tabulaire stocke explicitement une valeur pour chaque couple (état, action), tandis que l'approche DQN repose sur une approximation paramétrique dont la complexité dépend de l'architecture du réseau plutôt que du nombre d'états.

C/ Apprentissage de la board1

Avec $\gamma = 0.95$ et une architecture MLP 16×16 sur *board1*, l'apprentissage est correct et converge de manière stable. La courbe de récompense cumulée présente une phase initiale fortement bruitée, caractéristique de l'exploration, puis une stabilisation rapide autour de valeurs comprises entre 0.5 et 0.6 pour la majorité des épisodes, indiquant l'acquisition d'une politique efficace malgré une exploration résiduelle.

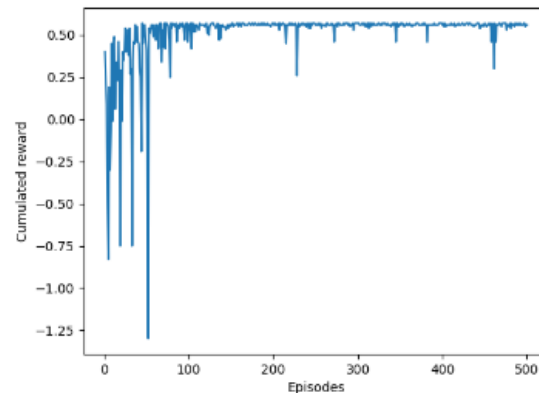
Les indicateurs de fin d'entraînement confirment cette convergence : à l'épisode 500, on observe un *Average Reward* de 0.5, un *Best Reward* et un *Last Reward* de 0.6, avec un taux d'exploration minimal atteint ($\epsilon = 0.05$). La perte finale est faible ($\text{Loss} \approx 2.66 \times 10^{-4}$), ce qui indique une optimisation stabilisée.

L'évaluation statistique confirme la qualité de la politique : sur 100 parties, le score moyen est d'environ 0.565, le taux de victoire est de 100 %, le nombre moyen de déplacements est de 3.54, et aucun passage sur une case poison n'est observé. L'agent termine ainsi systématiquement le jeu via des trajectoires courtes et sûres.

Si l'on modifie la position d'un poison, il est nécessaire de ré-apprendre, ou au minimum de ré-entraîner significativement, le réseau. La représentation d'état $[X, Y, F_i]$ ne contient pas d'information explicite sur la géométrie globale du plateau ni sur la localisation des dangers : déplacer un poison modifie donc la structure

des récompenses associée aux trajectoires possibles, rendant la fonction $Q(s, a)$ apprise incohérente avec le nouvel environnement. La même remarque s'applique à un changement de position de la maison, qui modifie la condition de terminaison et les trajectoires optimales sans être encodé dans l'état.

GAMMA 0.95
Neurones 16 16
Score moyen : 0.5646000000000001 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 3.54
Nombre moyen de cases poison par partie 0.0
board used: board1



D/ Apprentissage de la board2 et impact de la taille des couches cachées

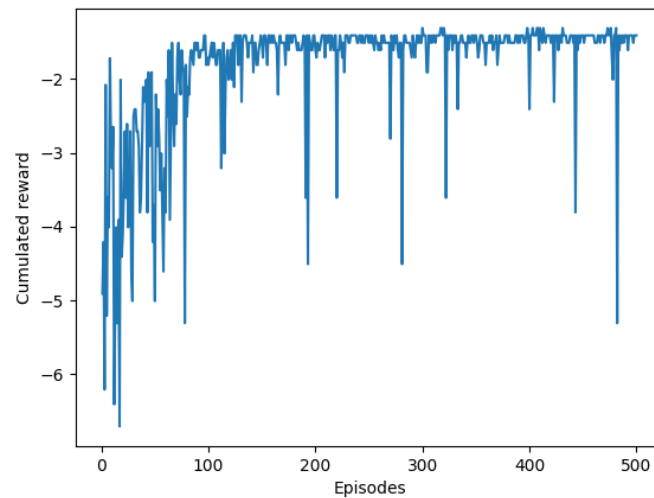
Sur *board2* avec $\gamma = 0.95$, l'apprentissage n'est pas correct lorsque l'on utilise un réseau de petite taille (MLP 3×3). Les résultats indiquent que l'agent ne parvient jamais à terminer la partie : le mode *stat* donne un taux de victoire de 0 %, un nombre moyen de déplacements égal à 50 (limite maximale imposée par la procédure de test), ainsi qu'un score moyen négatif (≈ -0.493). Le journal d'entraînement confirme ce comportement, avec l'indication récurrente "Max step atteint" et une récompense cumulée finale de -1.4 à l'épisode 500. La politique apprise ne conduit donc pas à l'état terminal et l'agent reste bloqué dans des trajectoires non concluantes jusqu'à atteindre la limite de pas.

Ce comportement est cohérent avec une capacité d'approximation insuffisante du réseau. Dans cette configuration plus complexe, l'état est continu (coordonnées X, Y normalisées) et la présence de plusieurs objectifs induit une fonction $Q(s, a)$ plus non linéaire, que le réseau 3×3 n'est pas capable de représenter de manière discriminante.

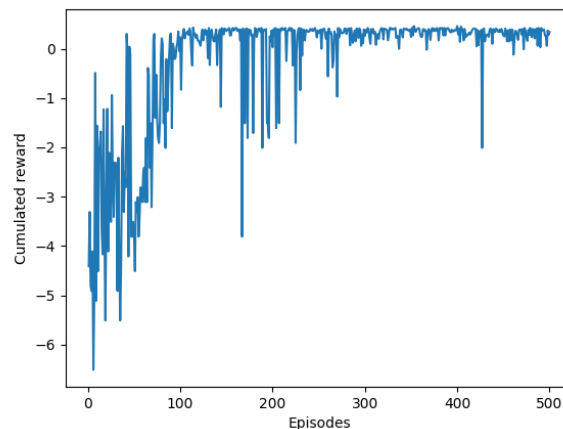
En augmentant la capacité du modèle avec une architecture 8×8 , l'apprentissage devient correct. À l'épisode 500, on obtient *Average Reward* = 0.2, *Best Reward* = 0.5 et *Last Reward* = 0.3. L'évaluation sur 100 parties indique un taux de victoire de 100 %, un score moyen d'environ 0.391, un nombre moyen de déplacements de 10.87 et aucun passage sur une case poison. Ces résultats montrent que l'échec observé précédemment provenait principalement d'une capacité de représentation insuffisante.

L'observation en mode *play* est cohérente : avec le réseau 3×3 , le Pacman ne parvient pas à identifier une séquence d'actions menant à la terminaison et boucle ou erre jusqu'à atteindre la limite de pas ; avec 8×8 , la politique devient suffisamment structurée pour collecter les objectifs et terminer la partie dans un nombre de pas modéré.

GAMMA 0.95
Neurones 3, 3
Score moyen : -0.4930000000000001 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.95
Neurones 8, 8
Score moyen : 0.3913 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.87
Nombre moyen de cases poison par partie 0.0
board used: board2



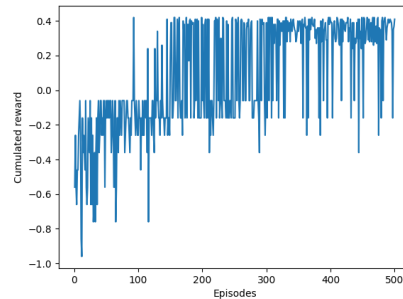
E/ Influence des hyperparamètres γ , MEM_SIZE et MAX_STEP

Les expériences ont été menées sur *board2* avec une architecture 16×16 , en faisant varier séparément γ , la taille du replay buffer (MEM_SIZE) et l'horizon maximal d'un épisode (MAX_STEP). Les courbes de reward cumulé et les statistiques finales mettent en évidence des rôles distincts de ces paramètres dans l'apprentissage d'une politique terminale.

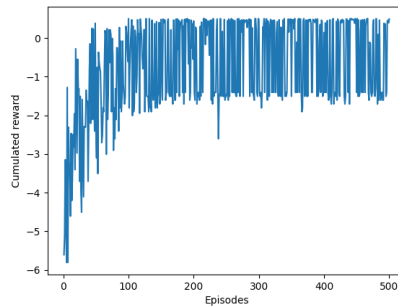
- Avec $\gamma = 0.95$, $MEM_SIZE = 4096$ et $MAX_STEP = 15$, la courbe présente un transitoire fluctuant puis une stabilisation vers des rewards positifs. Les statistiques confirment une politique gagnante (100 % de victoires, 10.93 déplacements moyens, 0 poison). Un horizon de l'ordre de 15 pas reste donc compatible avec l'atteinte de l'objectif sur *board2*.
- Avec $\gamma = 0.5$, $MEM_SIZE = 4096$ et $MAX_STEP = 150$, la courbe reste très bruitée et la politique est instable (57 % de victoires, 28.54 déplacements moyens). Cette dégradation est cohérente avec une actualisation forte des récompenses futures, qui rend l'agent plus myope et moins apte à structurer systématiquement des trajectoires complètes menant à la terminaison.
- Avec $\gamma = 0.95$, $MEM_SIZE = 409$ et $MAX_STEP = 150$, la convergence demeure lisible et les performances restent comparables (100 % de victoires, ≈ 10.95 déplacements moyens, 0 poison). Dans ce contexte, la réduction de la mémoire ne constitue pas un facteur limitant majeur.
- Avec $MAX_STEP = 10$ (et $MEM_SIZE = 409$), l'apprentissage échoue pour $\gamma = 0.5$ comme pour $\gamma = 0.95$: "Max step atteint" apparaît systématiquement et le taux de victoire tombe à 0 %, avec un nombre moyen de déplacements égal au plafond de test. Un horizon de 10 pas est donc insuffisant pour permettre l'atteinte régulière de l'état terminal, rendant la récompense de victoire pratiquement inobservable et empêchant l'apprentissage, indépendamment de γ .

Globalement, γ est déterminant pour structurer les trajectoires à long terme, tandis que MAX_STEP constitue une contrainte d'apprenabilité : en dessous d'un certain seuil, l'apprentissage d'une politique terminale devient impossible faute de transitions informatives. À l'inverse, la taille du replay buffer a ici un impact plus limité tant que l'agent observe suffisamment souvent des trajectoires terminales.

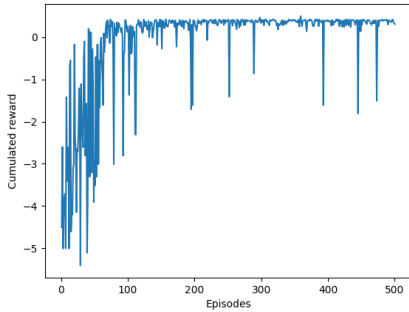
GAMMA 0.95
Neurones 16*16
MAX STEP 15
BUFFER 4096
Score moyen : 0.3697 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.93
Nombre moyen de cases poison par partie 0.0
board used: board2



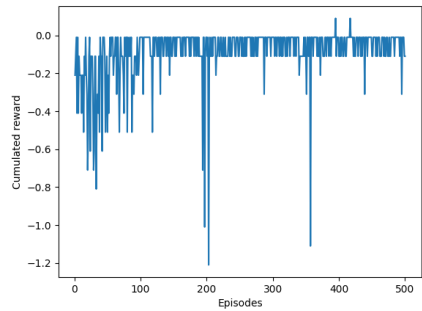
GAMMA 0.5
Neurones 16*16
MAX STEP 150
BUFFER 4096
Score moyen : 0.0885999999999993 Pourcentage victoire : 57.0 %
Nombre de déplacements moyens 28.54
Nombre moyen de cases poison par partie 0.0
board used: board2



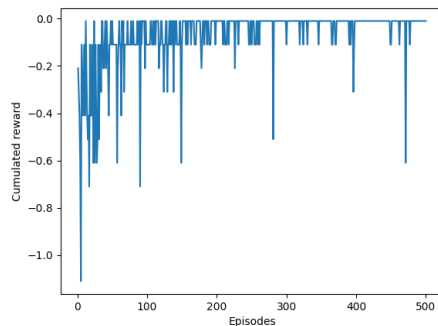
GAMMA 0.95
Neurones 16*16
MAX STEP 150
BUFFER 409
Score moyen : 0.3904999999999996 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 10.95
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.5
Neurones 16*16
MAX STEP 10
BUFFER 409
Score moyen : -0.42200000000000015 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2



GAMMA 0.95
Neurones 16*16
MAX STEP 10
BUFFER 409
Score moyen : -0.40000000000000002 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board2



F/ Apprentissage de la board 3

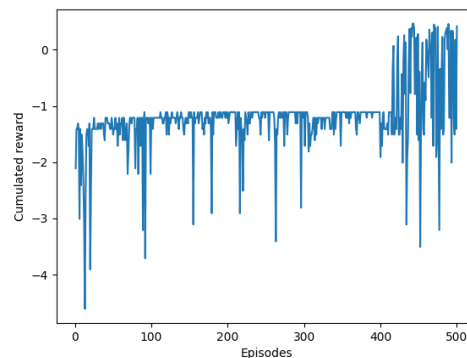
La complexification du jeu avec *board3* permet d'analyser plus finement l'adéquation entre la capacité du réseau et la difficulté de la tâche. Avec les paramètres classiques et une architecture 16×16 , les performances restent insuffisantes : score moyen négatif (≈ -0.235), taux de victoire limité à 26 % et 43.44 déplacements moyens, sans passage sur les poisons. Les courbes de reward cumulé restent majoritairement négatives, avec quelques épisodes favorables isolés, indiquant que l'agent découvre ponctuellement des trajectoires gagnantes sans stabiliser une politique terminale fiable. Le réseau apparaît sous-capacitaire pour approximer efficacement $Q(s, a)$ dans cet environnement plus complexe.

La réduction de la taille du replay buffer, qui restait compatible avec *board2*, ne se transpose pas à *board3*. Avec $MEM_SIZE = 409$ et un réseau 16×16 , les performances se dégradent fortement (0 % de victoires, score moyen ≈ -0.497 , 50 déplacements moyens). La courbe reste centrée sur des valeurs négatives sans tendance claire à la convergence, suggérant une diversité d'expériences insuffisante pour apprendre une politique terminale.

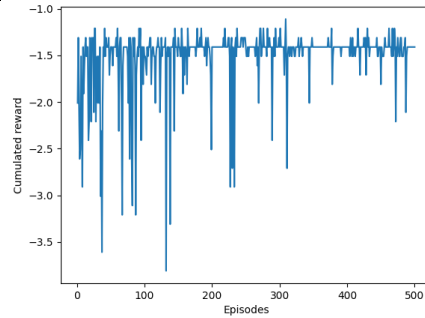
L'augmentation de la capacité du réseau améliore nettement les performances jusqu'à un certain seuil. Avec une architecture 32×32 , l'apprentissage devient pleinement opérationnel (100 % de victoires, score moyen ≈ 0.501 , 39.73 déplacements moyens). La courbe présente une transition claire vers un régime positif et un plateau relativement stable, indiquant une capacité d'approximation suffisante.

Une augmentation supplémentaire (128×128) ne conduit pas à une amélioration systématique : le taux de victoire reste élevé (95 %) et le score moyen ≈ 0.482 , sans gain net par rapport à 32×32 . Les courbes sont qualitativement proches, suggérant qu'au-delà d'un certain seuil, l'augmentation du nombre de paramètres n'apporte plus de bénéfice significatif et peut accroître la variance ou la difficulté d'optimisation. Dans ce cadre, 32×32 constitue un compromis pertinent entre expressivité et stabilité.

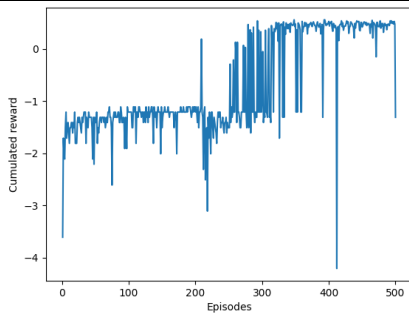
Paramètres classiques
Neurones 16*16
Score moyen : -0.23540000000000014 Pourcentage victoire : 26.0 %
Nombre de déplacements moyens 43.44
Nombre moyen de cases poison par partie 0.0
board used: board3



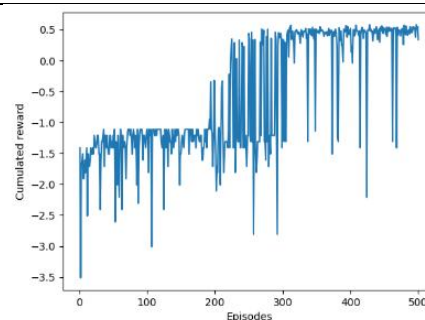
BUFFER 409
Neurones 16*16
Score moyen : -0.4970000000000001 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board3



Paramètres classiques
Neurones 32*32
Score moyen : 0.5007 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 39.73
Nombre moyen de cases poison par partie 0.0
board used: board3



Paramètres classiques
Neurones 128*128
Score moyen : 0.48229999999999995 Pourcentage victoire : 95.0 %
Nombre de déplacements moyens 33.77
Nombre moyen de cases poison par partie 0.0
board used: board3



G/ Double DQN et taille du réseau

Le passage à Double DQN vise à corriger un défaut du DQN standard : le biais de sur-estimation induit par l'opérateur max dans la cible de Bellman. Le principe consiste à dissocier la sélection de l'action future (réseau *online*) et son évaluation (réseau *target*). Cette modification améliore en général la stabilité de l'apprentissage, sans supprimer pour autant les contraintes liées à la capacité du modèle et à la difficulté intrinsèque de la tâche.

Pour passer de DQN à Double DQN, la cible TD a été modifiée afin de dissocier la sélection et l'évaluation de l'action future. L'action a^* est sélectionnée à l'aide du réseau online Q_θ , puis évaluée avec le réseau target Q_{θ^-} . La cible devient alors $y = r + \gamma(1 - d) Q_{\theta^-}(s', \arg \max_{a'} Q_\theta(s', a'))$. Le code correspondant est fourni en annexe (modification de la méthode `learn()`).

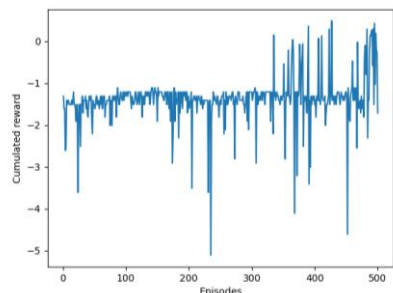
Sur *board3*, les résultats montrent que la convergence demeure fortement dépendante de la taille du réseau. Avec une architecture 16×16 , la courbe reste globalement négative et bruitée, sans transition durable vers un régime positif. Les statistiques confirment l'échec : score moyen ≈ -0.249 , 0 % de victoires et 50 déplacements moyens (plafond de test). Le nombre moyen de poisons restant nul indique l'acquisition de comportements d'évitement, mais sans organisation d'une trajectoire complète menant à la maison après consommation des nourritures.

Avec une architecture 32×32 , la dynamique s'améliore : la courbe présente une progression plus visible et l'apparition d'épisodes positifs plus fréquents, ce qui se traduit par une amélioration partielle (62 % de victoires, score moyen ≈ 0.1485). Toutefois, la politique reste instable et le nombre de déplacements demeure élevé (44.15).

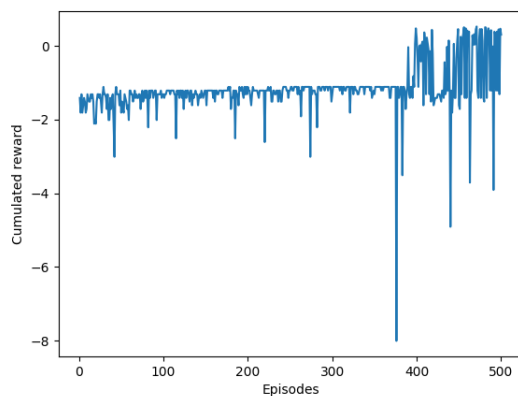
Avec une architecture 64×64 , l'apprentissage devient fiable. La courbe montre une transition nette vers un plateau positif, malgré quelques chutes ponctuelles, et l'évaluation atteint 100 % de victoires avec un score moyen ≈ 0.5058 et 39.42 déplacements moyens, toujours sans passage sur les poisons. Cela suggère que sur *board3*, la difficulté principale n'est pas uniquement la sur-estimation corrigée par Double DQN, mais également la nécessité d'une capacité d'approximation suffisante pour représenter $Q(s, a)$ dans un espace d'états plus riche et avec des dépendances temporelles plus longues.

Ces expériences indiquent ainsi que Double DQN constitue une amélioration algorithmique pertinente pour limiter certains effets d'instabilité, mais que ses bénéfices restent conditionnés par un dimensionnement adéquat du réseau. Sur *board3*, l'échec en 16×16 et la progression observée en 32×32 puis 64×64 montrent que l'augmentation de la capacité du modèle reste un levier déterminant, Double DQN jouant principalement un rôle de stabilisation.

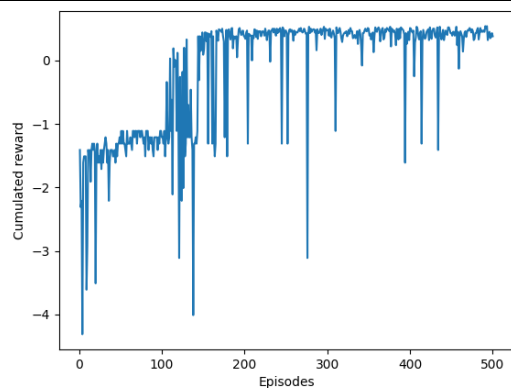
Paramètres classiques
Neurones 16*16
Score moyen : -0.24900000000000003 Pourcentage victoire : 0.0 %
Nombre de déplacements moyens 50.0
Nombre moyen de cases poison par partie 0.0
board used: board3



Paramètres classiques
Neurones 32*32
Score moyen : 0.14849999999999997 Pourcentage victoire : 62.0 %
Nombre de déplacements moyens 44.15
Nombre moyen de cases poison par partie 0.0
board used: board3



Paramètres classiques
Neurones 64, 64
Score moyen : 0.5058 Pourcentage victoire : 100.0 %
Nombre de déplacements moyens 39.42
Nombre moyen de cases poison par partie 0.0
board used: board3



ANNEXE :

Code de la fonction learn du double DQN :

```
def learn(self, gamma):
    if self.memory.mem_count < BATCH_SIZE:
        return 0

    self.learn_step_counter += 1

    states, actions, rewards, states_, dones = self.memory.sample()

    states = torch.tensor(states, dtype=torch.float32).to(DEVICE)
    actions = torch.tensor(actions, dtype=torch.long).to(DEVICE)
    rewards = torch.tensor(rewards, dtype=torch.float32).to(DEVICE)
    states_ = torch.tensor(states_, dtype=torch.float32).to(DEVICE)
    dones = torch.tensor(dones, dtype=torch.float32).to(DEVICE)

    batch_indices = torch.arange(BATCH_SIZE, device=DEVICE)

    #  $Q(s,a)$  prédite par le réseau online
    q_values = self.network(states)
    q_pred = q_values[batch_indices, actions]

    # ----- Double DQN target -----
    with torch.no_grad():
        # 1) sélection de l'action au prochain état via ONLINE network
        q_next_online = self.network(states_) # shape (B, A)
        next_actions = torch.argmax(q_next_online, dim=1) # shape (B,)

        # 2) évaluation de cette action via TARGET network
        q_next_target = self.target_network(states_) # shape (B, A)
        q_next_eval = q_next_target[batch_indices, next_actions] # shape (B,)

        # 3) cible TD
        q_target = rewards + gamma * q_next_eval * dones
    # -----

    loss = self.network.loss(q_pred, q_target)

    self.network.optimizer.zero_grad()
    loss.backward()
    self.network.optimizer.step()
```



```
if self.learn_step_counter % TARGET_UPDATE_FREQ == 0:  
    self.target_network.load_state_dict(self.network.state_dict())  
  
return loss.item()
```